Guidelines for Writing C++ Programs

Amruth N. Kumar, amruth@ramapo.edu

Language Details - Strictly adhere to these guidelines

Readability:

- List the members in a class in the following order:
  - `public` at the top, followed by `protected` and finally `private` members.
  - List members in the following order: constants, variables, constructor(s), destructor, selector(s), mutator(s), utility functions.
- Use mnemonic variable and function names. Do **not** use inscrutable abbreviations for names.
- Properly indent your code.
- Comment your code. Include the pseudocode as comments in the program.

Reliability:

- **Class Definition:**
  - Avoid making data members `public` in the class definition.
- **Constructor:**
  - Make your constructors robust.
  - Be sure to define a default constructor: a constructor that can be called without any parameters.
  - If you have aggregate data members in your class: arrays, structures or other objects, be sure to define a copy constructor.
- **Selectors:**
  - Make them `const` functions.
  - Do not break encapsulation: when *returning* an array, pointer to a structure, or reference to an object from a selector, be sure to return a copy and not a reference to the original.
- **Mutators:**
  - They must be robust, i.e., they should not permit invalid values (e.g., 32 for day of the month) to be assigned to data members.
  - They must return an error value: to indicate whether the last parameter that was passed was used to change the value of a data member or not.
  - They must **not** read values from the keyboard, but rather accept them as parameters read and passed from the client function.
  - Do not break encapsulation: when arrays, structures or objects are passed as parameters, make local copies of them, don't create a local reference to the passed parameters.
- **Client Code:**

- Make your inputs robust. E.g., if you are reading the value of a month, do not accept any value other than 1-12.
- When using functions, pass parameters by reference only where necessary. Pass parameters by value otherwise.

Modifiability:

- Use symbolic constants where possible. Do not use any magic numbers in your code.
- Do NOT use any global variables. None whatsoever.

Reusability:

- Use principles of modularity - use a header and an implementation file for each class. Put client code into a separate file.
- Avoid Input/Output operations in member functions of your model classes, Put them in the client code or view classes instead.

Efficiency:

- Use inline functions where applicable, for efficiency.

  Use member initializer syntax for the same reason in constructors, whenever possible.
- When using inheritance, make base class data members protected.

Ease of Use:

- Format the output of your program to be easily understandable.

Class Design:

- **Completeness:** Include all necessary data members and member functions in a class. Do not define client functions for chores that must be handled by a class.
- **Cohesiveness:** Only those data members must be included which are naturally a part of the entity being modeled. Do not include unnecessary / irrelevant members in a class.

Readability

It is very important to write clear, readable code. 20% of your grade is set aside for readability. Readability is inversely proportional to the time it takes to read a program and understand how it works: the more the time I spend trying to understand your code, the fewer the points you get for readability.

You can improve readability of your code by:

- Decomposing your problem top-down and writing functions to closely mirror this decomposition
- Providing adequate documentation at each stage for what your code does
- Using reasonable variable/function names, etc.